Höhere technische Bundeslehranstalt
und Bundesfachschule
im Hermann Fuchs Bundesschulzentrum

AGV Control Center

Diplomarbeit

Fachrichtung, Schulautonomer Schwerpunkt
Elektronik und technische Informatik, Coding AI

ausgeführt im Schuljahr 2026/2027 von:

Andreas Kastner, 5AHELS

Florian Wohlfarter, 5AHELS

Betreuer:

Dipl.-Ing. Lukas Lämpel

4. November 2026

Eidesstattliche Erklärung

Wir erklären an Eides statt, dass wir die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als angegebene Quellen und Hilfsmittel nicht direkt benutzt und die benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht haben.

Braunau am Inn, 04.11.2026	Andreas Kastner	_____
Ort, Datum	*Name*	*Unterschrift*

Braunau am Inn, 04.11.2026	Florian Wohlfarter	_____
Ort, Datum	*Name*	*Unterschrift*

Abstract

Das Abstract ist eine prägnante Zusammenfassung der gesamten Arbeit in englischer Sprache. Es umfasst typischerweise 150–250 Wörter und beschreibt das Problem, die verwendete Methodik, die wichtigsten Ergebnisse und die Schlussfolgerungen. Das Abstract ermöglicht Lesenden eine schnelle Einschätzung der Relevanz der Arbeit.

Example

Modern software development faces significant challenges in managing dependencies across large-scale projects, with version conflicts consuming substantial developer time. Existing dependency management tools provide automatic updates but fail to consider semantic compatibility at the API level, leading to runtime errors and costly hotfixes.

This thesis presents an intelligent dependency analyzer that leverages static code analysis to detect actual API usage patterns and automatically suggest compatible library versions. The system combines abstract syntax tree analysis with a machine learning classifier trained on 50,000 real-world dependency conflicts.

Evaluation on ten open-source projects (10,000–100,000 lines of code) demonstrates that the analyzer identifies 94 % of breaking changes before integration, with an average analysis time of 3.2 minutes per project. The false positive rate was reduced to 8 % compared to 23 % in existing tools.

The proposed solution significantly reduces integration risks and accelerates development cycles. Future work will extend support to dynamic languages and incorporate runtime behavior analysis.

Kurzfassung

Die Kurzfassung ist eine prägnante Zusammenfassung der gesamten Arbeit in deutscher Sprache. Sie umfasst typischerweise 150–250 Wörter und beschreibt das Problem, die verwendete Methodik, die wichtigsten Ergebnisse und die Schlussfolgerungen. Die Kurzfassung ermöglicht Lesenden eine schnelle Einschätzung der Relevanz der Arbeit.

Beispiel

Die moderne Softwareentwicklung steht vor erheblichen Herausforderungen bei der Verwaltung von Abhängigkeiten in großangelegten Projekten, wobei Versionskonflikte erhebliche Entwicklerzeit binden. Bestehende Dependency-Management-Tools bieten zwar automatische Updates, berücksichtigen jedoch nicht die semantische Kompatibilität auf API-Ebene, was zu Laufzeitfehlern und kostspieligen Hotfixes führt.

Diese Diplomarbeit präsentiert einen intelligenten Dependency-Analyzer, der mittels statischer Code-Analyse tatsächliche API-Nutzungsmuster erfasst und automatisch kompatible Bibliotheksversionen vorschlägt. Das System kombiniert Abstract-Syntax-Tree-Analyse mit einem Machine-Learning-Klassifikator, der mit 50.000 realen Abhängigkeitskonflikten trainiert wurde.

Die Evaluierung an zehn Open-Source-Projekten (10.000–100.000 Zeilen Code) zeigt, dass der Analyzer 94 % der Breaking Changes vor der Integration identifiziert, bei einer durchschnittlichen Analysezeit von 3,2 Minuten pro Projekt. Die False-Positive-Rate wurde gegenüber bestehenden Tools von 23 % auf 8 % reduziert.

Die vorgeschlagene Lösung reduziert Integrationsrisiken signifikant und beschleunigt Entwicklungszyklen. Zukünftige Arbeiten werden die Unterstützung dynamischer Sprachen erweitern und Laufzeitverhaltensanalyse integrieren.

Inhaltsverzeichnis

Beispielhafte Inhalte: Zitieren, Abbildungen, Quelltext

Hinweis

Achtung!

In diesem Kapitel ist nicht Teil der Diplomarbeit, sondern dient lediglich der Veranschaulichung, wie Abbildungen, Zitate und Quelltext zu verwenden sind.

Die Zitate, Abbildungen und Listings werden automatisch in das Quellen- und Abbildungsverzeichnis übernommen. Das Literatur-, Abbildungs- und Listingsverzeichnis sind am Ende der Arbeit zu finden.

Auch beschrieben wird, wie mit KI-generierten Inhalten umzugehen ist. Alle KI-generierten Inhalte müssen als solche gekennzeichnet werden, um Transparenz zu gewährleisten.

Abbildungen

Abbildung sind mit einer Abbildungsnummer und einer Unterschrift zu versehen die kurz die Abbildung beschreibt. Abbildungen gehören zum umgebenden Text und müssen dort erwähnt werden.

Beispiel

Beispiel: In Abbildung **??** wird das Logo der HTL Braunau dargestellt.

Wird auf eine Abbildung referenziert die sich weiter von der aktuellen Textstelle entfernt befindet so kann auch die Seitennummer hinzugefügt werden.

Beispiel

Beispiel: Siehe Abbildung **??** auf Seite **??**.



Abbildung 1: Logo der HTL Braunau



(a) Logo der HTL Braunau.



(b) Logo der HTL Braunau.

Abbildung 2: Zwei Logos der HTL Braunau in einer Abbildung zusammengefasst



(a) Logo



(b) Logo



(c) Logo

Abbildung 3: Dreimal das Logo der HTL Braunau

Zitate, Quellen, Fußnoten

Zitation dient der Nachvollziehbarkeit und ist verpflichtend, sobald fremde Gedanken, Daten oder Formulierungen verwendet werden. Verwenden Sie im gesamten Dokument eine einheitliche Zitierweise (z. B. numerische Zitate in eckigen Klammern) und verweisen Sie immer auf das Literaturverzeichnis.

Allgemeine Regeln

Kurzregeln:

- Direkte Zitate sind selten und müssen wörtlich übernommen werden.
- Paraphrasen sind bevorzugt, aber ebenfalls zu belegen.
- Längere Zitate werden als Blockzitat gesetzt.
- Verkürzungen in Zitaten werden mit [...] gekennzeichnet.

Beispiel für Quellenangabe

Quellenangaben gehören immer ins Literaturverzeichnis, das automatisch aus den verwendeten Zitationen erstellt wird [?].

Beispiel für Quellenangabe

Das Verwenden fremden Gedankenguts ohne Quellenangabe ist ein Plagiat und kann rechtliche Folgen haben [?].

Praktische Beispiele

Längeres Zitat mit Verkürzung und anschließender Einordnung

Navigation in Single-Page-Websites

> *„We tend to think of navigating a website as clicking from page-to-page via some kind of global navigation [...] When it comes to a single page, we often think scrolling is the one and only way to move from one end to the next.“* [?]

Der Autor betont, dass Nutzer:innen Navigation oft mit klassischen Menüs gleichsetzen. Für unser Projekt bedeutet das, dass bei One-Page-Designs klare Orientierungspunkte (z. B. Abschnittsmarker oder Kurz-Links) notwendig sind, um den roten Faden der Nutzung zu sichern.

Kurzregeln zu Footnotes vs. Zitation:

- Zitate verwenden, wenn eine Aussage, Zahl oder Idee aus einer Quelle stammt und nachprüfbar belegt werden muss.
- Fußnoten verwenden, wenn eine kurze Nebeninformation, ein Begriffshinweis oder eine ergänzende Erklärung den Lesefluss sonst stören würde.

Kurze Nebeninformationen als Fußnote

Die Implementierung erfolgte in Python 3.11[1], da diese Version zum Zeitpunkt der Entwicklung die aktuellste stabile Version war.

Begriffserklärung als Fußnote

In diesem Abschnitt wird der Begriff *NPR* verwendet.[2]

[1] Verwendet wurde die offizielle Distribution von python.org, Version 3.11.4
[2] NPR steht für *Non-photorealistic Rendering* und beschreibt Techniken, die bewusst nicht fotorealistisch

Methodischer Hinweis als Fußnote

Die Messwerte wurden in einer Testumgebung erhoben.[3]

Künstliche Intelligenz in akademischen Arbeiten

Der Einsatz von KI-Tools (wie Large Language Models, Bildgeneratoren oder Code-Assistenten) nimmt in akademischen Arbeiten zu. Diese Tools können bei Recherche, Ideenfindung, Code-Generierung und Texterstellung unterstützen. **Wichtig:** Jeder Abschnitt oder jede Information, die mit Hilfe von KI erstellt oder wesentlich überarbeitet wurde, muss als **Fußnote** gekennzeichnet werden. Dies garantiert Transparenz und ermöglicht es dem Leser, die Quellenqualität einzuschätzen.

Die Fußnote sollte folgende Informationen enthalten:

- Verwendetes Modell/Tool (z.B. *Claude Haiku 4.5*, *ChatGPT-4*, *GitHub Copilot*)
- Datum der Anfrage
- Kurze Beschreibung des Prompts/der Anfrage
- Bearbeitung durch den Autor (falls zutreffend)

Beispiel für Absatz mit KI-Unterstützung

KI-Systeme revolutionieren die Art und Weise, wie wir Informationen verarbeiten und Entscheidungen treffen. Durch ihre Fähigkeit zur Mustererkennung und Datenanalyse ermöglichen sie schnellere und präzisere Lösungen in verschiedensten Anwendungsbereichen.[4]

Diese Integration von KI in wissenschaftliche Arbeiten erfordert aber kritisches Denken und Verantwortung. Der Autor bleibt für alle verwendeten Inhalte verantwortlich, auch wenn diese mit KI-Unterstützung erstellt wurden.

Beispiel für Absatz mit KI-Unterstützung

KI-Systeme revolutionieren die Art und Weise, wie wir Informationen verarbeiten und Entscheidungen treffen. Durch ihre Fähigkeit zur Mustererkennung und Datenanalyse ermöglichen sie schnellere und präzisere Lösungen in verschiedensten Anwendungsbereichen.[5]

Diese Integration von KI in wissenschaftliche Arbeiten erfordert aber kritisches Denken und Verantwortung. Der Autor bleibt für alle verwendeten Inhalte verantwortlich, auch wenn

wirken.

[3] Tests auf isoliertem System ohne Produktivlast; Ergebnisse nur bedingt auf Live-Betrieb übertragbar.

[4] Dieser Absatz basiert auf einer Anfrage an Claude Haiku 4.5 vom 12.02.2026 mit dem Prompt: „Schreibe einen einleitenden Absatz über die Auswirkungen von KI auf Informationsverarbeitung und Entscheidungsfindung.“ Das Ergebnis wurde für akademische Zwecke angepasst und auf Korrektheit überprüft.

[5] Dieser Absatz basiert auf einer Anfrage an Claude Haiku 4.5 vom 12.02.2026 mit dem Prompt: „Schreibe einen einleitenden Absatz über die Auswirkungen von KI auf Informationsverarbeitung und Entscheidungsfindung.“ Das Ergebnis wurde für akademische Zwecke angepasst und auf Korrektheit überprüft.

diese mit KI-Unterstützung erstellt wurden.

Listings, Code

Quelltext ist ein wesentlicher Bestandteil einer Informatik Diplomarbeit. Listings werden wie Abbildungen nummeriert und mit einer Unterschrift versehen. Ebenfalls müssen sie im Text referenziert werden.

Beispiele

Listing **??** auf Seite **??** zeigt ein Hallo Welt Programm.

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

void main()
{
  printf ( "Hallo!");
  getch ();

  system ("cls"); // Kommentar

  printf ("Hello World!");
  getch ();
}
```

Listing 1: Mein erstes C-Programm.

Das PHP Programm **??** dient zum ermitteln aller Dateien eines Unterverzeichnisses.

```
<?php
if ($handle = opendir(realpath(__DIR__ . '/../userimages'))) {

    while (false !== ($file = readdir($handle))) {
        echo "$file//";
    }

    closedir($handle);
}
?>
```

Listing 2: Alle Dateinamen ausgeben.

```
// Calculates the mouse position.
function getMousePos(canvas, evt) {
  var rect = canvas.getBoundingClientRect();
  return {
    x: evt.clientX - rect.left,
    y: evt.clientY - rect.top
  };
}
```

Listing 3: Mausposition ermitteln.

```css
/* Zugriff auf ein HTML Tag */
body {
  background-color: blue;
}

/* Zugriff auf eine Klasse */
.class {
  font-weight: bold;
}

/* Zugriff auf eine ID */
#id {
  color: red;
}

/* Zugriff auf eine Pseudoklasse */
#id:nth-child(@number*3-2) {
  color: blue;
}
```

Listing 4: Kurzer CSS–Quelltext.

```html
<html>
  <head>
    <title>Test HTML Einbindung</title>
    <style type="text/css">
      #content {
        width: 500px;
        margin: 0 auto;
      }
    </style>
  </head>
  <body>
    <div id="content">
      <!-- Content goes here -->
    </content>
  </body>
</html>
```

Listing 5: HTML–Quelltext

```javascript
var gIntervalId = window.setInterval ( "checkPos();", 5000 );

function success(position)
{
  var s = document.querySelector("#status");

  if (s.className == "success")
  {
      // Kommentar
      return;
    }
}
```

Listing 6: JavaScript–Quelltext

1 Einleitung[AK]

In diesem Kapitel werden die Ziele dieser Diplomarbeit festgehalten. Die Gliederung der Arbeit wird durch einen kurzen Überblick über die Kapitel beschrieben.

1.1 Motivation, Problemstellung und Zielsetzung

Dieser Abschnitt erläutert die Motivation für die Themenwahl, beschreibt das zu lösende Problem und definiert die konkreten Ziele der Arbeit. Eine klare Problemstellung und explizite Zielsetzung sind wesentlich, um den Rahmen der Arbeit abzustecken und die Erfolgskriterien zu definieren.

Beispiel

Motivation

In der modernen Softwareentwicklung stellt die Verwaltung von Abhängigkeiten in großen Projekten eine zunehmende Herausforderung dar. Entwicklerteams verbringen durchschnittlich 20 % ihrer Arbeitszeit mit der Analyse und Behebung von Versionskonflikten zwischen Bibliotheken.
...

Problemstellung

Bestehende Dependency-Management-Tools bieten zwar automatische Aktualisierungen an, berücksichtigen jedoch nicht die semantische Kompatibilität auf API-Ebene. Dies führt zu Breaking Changes, die erst zur Laufzeit erkennbar werden und kostspielige Hotfixes erfordern.
...

Zielsetzung

Ziel dieser Arbeit ist die Entwicklung eines intelligenten Dependency-Analyzers, der mittels statischer Code-Analyse die tatsächliche API-Nutzung erfasst und automatisch kompatible Versionen vorschlägt. Das System soll mindestens 90 % der Breaking Changes bereits vor der Integration erkennen und die Analysezeit unter 5 Minuten für typische Projekte halten.
...

1.2 Gliederung der Arbeit

Dieser Abschnitt gibt einen Überblick über den Aufbau der Arbeit und beschreibt kurz den Inhalt der einzelnen Kapitel. Dies ermöglicht Lesenden eine schnelle Orientierung und zeigt die logische Struktur der Argumentation auf.

Beispiel

Die vorliegende Arbeit gliedert sich in sieben Kapitel:

Kapitel 1 – Einleitung führt in die Thematik ein, beschreibt die Problemstellung und definiert die Zielsetzung der Arbeit.

Kapitel 2 – Grundlagen vermittelt das notwendige theoretische Fundament, definiert zentrale Begriffe und stellt den aktuellen Stand der Technik dar.

Kapitel 3 – Konzepte präsentiert die entwickelten Lösungsansätze, beschreibt die Systemarchitektur und begründet getroffene Designentscheidungen.

Kapitel 4 – Implementierung dokumentiert die technische Umsetzung, erläutert kritische Implementierungsdetails und beschreibt verwendete Algorithmen.

Kapitel 5 – Prototyp und Evaluation stellt den entwickelten Prototyp vor, beschreibt durchgeführte Tests und evaluiert die Ergebnisse hinsichtlich der definierten Zielsetzung.

Kapitel 6 – Zusammenfassung und Ausblick fasst die wichtigsten Erkenntnisse zusammen, reflektiert kritisch die Ergebnisse und skizziert mögliche Weiterentwicklungen.

Kapitel 7 – Projektmanagement dokumentiert die Projektorganisation, Aufgabenverteilung und den zeitlichen Ablauf der Diplomarbeit.

2 Grundlagen[FW]

2.1 Begriffsdefinitionen

In diesem Abschnitt werden alle fachspezifischen Begriffe und Konzepte definiert, die für das Verständnis dieser Arbeit notwendig sind.

Beispiel

Neuronales Netzwerk
Ein künstliches neuronales Netzwerk ist ein mathematisches Modell, das aus miteinander verbundenen Knoten (Neuronen) besteht, die in Schichten angeordnet sind. Jede Verbindung besitzt ein Gewicht, das während des Trainings angepasst wird, um komplexe nichtlineare Zusammenhänge zwischen Eingabe- und Ausgabedaten zu lernen.

2.2 State of the art

In diesem Abschnitt wird der aktuelle Stand der Technik oder Forschung im relevanten Fachgebiet dargestellt.

Beispiel

Tesla Autopilot
Teslas Fahrassistenzsystem verwendet acht Kameras, zwölf Ultraschallsensoren und einen Radar zur Umgebungserfassung. Das neuronale Netzwerk wurde mit Milliarden von Kilometern realer Fahrdaten trainiert und ermöglicht adaptive Geschwindigkeitsregelung, Spurhalten und automatisches Einparken.

2.3 Theoretische Grundlagen

Dieser Abschnitt erläutert die theoretischen Konzepte, Modelle, Algorithmen oder mathematischen Methoden, die dem entwickelten System zugrunde liegen.

Beispiel

Diskrete Kosinustransformation (DCT)

Die DCT ist eine lineare Transformation, die ein Signal vom Ortsbereich in den Frequenzbereich überführt. Für ein eindimensionales Signal $x[n]$ der Länge N ist die DCT definiert als:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cos\left[\frac{\pi}{N}\left(n + \frac{1}{2}\right)k\right] \tag{2.1}$$

Die DCT konzentriert die Energie des Signals in wenigen niedrigen Frequenzkoeffizienten, was effiziente Kompression ermöglicht. Sie bildet die Grundlage des JPEG-Algorithmus.

2.4 Technologische Grundlagen

In diesem Abschnitt werden die im Projekt verwendeten Technologien, Programmiersprachen, Frameworks, Protokolle oder Hardware-Komponenten vorgestellt.

Beispiel

React (Frontend-Framework)

React ist eine JavaScript-Bibliothek zur Erstellung von Benutzeroberflächen, entwickelt von Meta (Facebook). Das Kernkonzept sind wiederverwendbare Komponenten, die ihren eigenen Zustand verwalten. Der Virtual DOM ermöglicht effiziente Updates, indem nur geänderte Teile der Benutzeroberfläche neu gerendert werden.

2.5 Voraussetzungen und Einschränkungen

Dieser Abschnitt definiert die Rahmenbedingungen, unter denen die Arbeit durchgeführt wurde, sowie bewusste Einschränkungen des Projektumfangs (Scope).

Beispiel

Plattform-Einschränkungen

Die Implementierung erfolgt ausschließlich für Desktop-Betriebssysteme (Windows 10/11, macOS 12+, Ubuntu 20.04+). Mobile Plattformen (iOS, Android) sind nicht Teil des Projektumfangs, da die Fokussierung auf Desktop-Systeme eine tiefere Integration mit Systemschnittstellen und ausführlichere Testszenarien ermöglicht.

3 Konzepte

Dieses Kapitel beschreibt die konzeptionellen Lösungsansätze für die zentrale Problemstellung der Arbeit. Ziel ist es, mehrere mögliche Architekturen und Vorgehensweisen nachvollziehbar darzustellen, ihre Vor- und Nachteile gegenüberzustellen und die Entscheidungsgrundlage für die gewählte Lösung zu liefern.

3.1 *Beispiel:* Hardware[AK]

3.1.1 Lösungsansatz 1

Hier wird der erste Lösungsansatz detailliert beschrieben, einschließlich der zugrunde liegenden Idee, der theoretischen Grundlagen und der erwarteten Vor- und Nachteile. Grafische Darstellungen wie Diagramme oder Skizzen unterstützen das Verständnis der Konzeptarchitektur.

Beispiel

Ansatz: Zentrale Server-basierte Verarbeitung
Alles wird auf einem Server verarbeitet. Der Client sendet Anfragen an einen zentralen Server, der alle Verarbeitungsschritte durchführt und Ergebnisse zurücksendet. Vorteil: Einfache Wartung und zentrale Kontrolle. Nachteil: Hohe Serverauslastung bei vielen gleichzeitigen Anfragen, Single Point of Failure.

...

3.1.2 Lösungsansatz 2

Dieser Abschnitt präsentiert einen alternativen Lösungsansatz mit dessen spezifischen Eigenschaften und Unterschieden zum ersten Konzept. Die Bewertung erfolgt anhand definierter Kriterien wie Machbarkeit, Kosten, Zeitaufwand und technischer Komplexität.

Beispiel

Ansatz: Verteilte Client-/Server-Architektur
Große Datenmengen werden vorab auf den Client übertragen und dort verarbeitet. Der Server bleibt stateless. Vorteil: Bessere Skalierbarkeit, offline-fähig, reduzierte Serverauslastung. Nachteil: Höherer Datentransfer initial, komplexere Client-side Logik, Browser-Speicherlimitierungen.

...

3.2 *Beispiel:* Frontend[AK]

3.2.1 Lösungsansatz 1

...

3.2.2 Lösungsansatz 2

...

3.3 *Beispiel:* Backend und Datenhaltung[FW]

3.3.1 Lösungsansatz 1

...

3.3.2 Lösungsansatz 2

...

3.4 Bewertung und Auswahl der Konzepte

In diesem Abschnitt werden die vorgestellten Konzepte systematisch anhand definierter Kriterien bewertet und verglichen. Basierend auf dieser Analyse wird begründet, welches Konzept als Grundlage für die Implementierung ausgewählt wurde. Die Darstellung macht die Entscheidungsfindung nachvollziehbar und dokumentiert die Gründe für die getroffene Auswahl.

Beispiel

Die Konzepte wurden anhand von fünf Kriterien bewertet: (1) Machbarkeit (Aufwand und technische Realisierbarkeit), (2) Performance (Verarbeitungsgeschwindigkeit und Ressourcenverbrauch), (3) Wartbarkeit (Code-Klarheit und Wartungsaufwand), (4) Skalierbarkeit (Wachstumsfähigkeit bei größeren Datenmengen), (5) Alignment mit bestehenden Standards.

...

Ergebnis der Bewertung: Konzept A zeigte die beste Balance zwischen Machbarkeit und Performance, während Konzept B zwar theoretisch überlegen war, aber erhebliche Implementierungszeitverluste mit sich brachte. Konzept C wurde aufgrund hoher Betriebskosten ausgeschlossen.

...

Entscheidung: Es wurde Konzept A als Grundlage für die Implementierung ausgewählt, da es das beste Kosten-Nutzen-Verhältnis bietet und zeitgerecht realisierbar ist. Optimierungen aus Konzept B können in zukünftigen Versionen integriert werden.

...

4 Implementierung

Dieses Kapitel dokumentiert die praktische Umsetzung der im Konzeptteil beschriebenen Lösung. Es werden die gewählte Architektur, zentrale Komponenten, verwendete Technologien sowie besondere Implementierungsentscheidungen und Herausforderungen nachvollziehbar dargestellt.

4.1 *Beispiel:* Hardware[AK]

4.1.1 Implementierungsdetails

Hier werden die konkrete technische Umsetzung der Systemkomponenten sowie wichtige Designentscheidungen beschrieben. Code-Snippets, UML-Diagramme und Erklärungen zu kritischen Algorithmen verdeutlichen die Implementierung.

Beispiel

Die Kernkomponente analysiert Eingabedaten durch einen dreistufigen Prozess: (1) Daten-Validierung mit JSON-Schema, (2) Transformation mittels einer Custom-Pipeline mit Pluggable-Architektur, (3) Ausgabe-Serialisierung in mehreren Formaten. Die Daten-Validierung wurde als Middleware implementiert, um Fehler früh zu erkennen. Besondere Aufmerksamkeit galt der Fehlerbehandlung mit aussagekräftigen Fehlermeldungen für End-User sowie detaillierten Logs für Developer.

...

4.1.2 Verwendete Technologien

Dieser Abschnitt listet alle eingesetzten Programmiersprachen, Frameworks, Bibliotheken und Tools auf und begründet deren Auswahl. Die Darstellung erfolgt strukturiert mit konkreten Versionsangaben und Einsatzzwecken.

Beispiel

Backend: Node.js 18.14.0 mit Express.js 4.18.2 für schnelle API-Entwicklung; Mongoose 7.0.0 für die MongoDB-Integration. Die Wahl fiel auf Node.js wegen der Performance und dem großen Ökosystem.

Frontend: React 18.2.0 mit TypeScript 4.9 für Type-Safety; Material-UI 5.11 für konsistentes Design. React wurde gewählt für Component-Reusability und Virtual DOM Performance.

...

4.2 *Beispiel:* Frontend[AK]

4.2.1 Implementierungsdetails

...

4.2.2 Verwendete Technologien

...

4.3 *Beispiel:* Backend und Datenhaltung[FW]

4.3.1 Implementierungsdetails

...

4.3.2 Verwendete Technologien

...

5 Prototyp und Ergebnisse[AK]

Dieses Kapitel stellt den entwickelten Prototyp vor und beschreibt die durchgeführten Tests sowie die erzielten Ergebnisse. Ziel ist es, die Funktionsfähigkeit der Lösung anhand konkreter Szenarien zu demonstrieren und die Resultate in Bezug auf die Zielsetzung zu bewerten.

5.1 *Beispiel:* Hardware[AK]

5.1.1 Beschreibung des Prototyps

Hier wird der entwickelte Prototyp mit seinen Funktionen, der Benutzeroberfläche und den umgesetzten Kernfunktionalitäten vorgestellt. Screenshots, Ablaufdiagramme und Beschreibungen der Benutzerinteraktion dokumentieren den aktuellen Entwicklungsstand.

Beispiel

Der Prototyp wurde als Web-Anwendung mit React (Frontend) und Node.js (Backend) implementiert. Die Benutzeroberfläche besteht aus drei Hauptkomponenten: (1) ein Eingabebereich für Konfigurationsparameter, (2) eine Visualisierungskomponente für Echtzeitdaten und (3) ein Export-Panel für Ergebnisse. Die Anwendung arbeitet vollständig offline und speichert Daten lokal in IndexedDB. Die Kernfunktionalitäten wurden durch 42 Unit-Tests validiert.

...

5.1.2 Tests und Evaluierung

Dieser Abschnitt beschreibt die durchgeführten Tests (Unit-Tests, Integrationstests, Benutzertests) sowie deren Ergebnisse. Die Evaluierung bewertet, ob die definierten Anforderungen erfüllt wurden und identifiziert Verbesserungspotenziale.

Beispiel

Folgende Testabdeckung wurde durchgeführt: (1) Unit-Tests für alle kritischen Funktionen mit 98,1 % Erfolgsquote, (2) Integrationstests mit realen Eingabedaten zeigten eine durchschnittliche Verarbeitungszeit von 2,3 Sekunden, (3) Usability-Tests mit 8 Testnutzern erreichten einen SUS-Wert von 72 Punkten. Browser-Kompatibilität wurde auf Chrome, Firefox und Safari getestet. Ein kritischer Bug in der Datenvalidierung wurde identifiziert und behoben.

...

5.1.3 Ergebnisse

Dieser Abschnitt fasst die wichtigsten Ergebnisse der Evaluierung zusammen. Die Resultate werden anhand quantitativer Metriken sowie qualitativer Beobachtungen dargestellt und in Bezug zu den in der Zielsetzung definierten Anforderungen bewertet.

Beispiel

Bei der Durchführung von insgesamt 42 Unit-Tests erreichte der Prototyp eine Erfolgsquote von 98,1 %. Die durchschnittliche Antwortzeit bei typischen Anwendungsfällen betrug 2,3 Sekunden, was die Anforderung von unter 5 Sekunden deutlich erfüllt.

...

5.2 *Beispiel:* Frontend[AK]

5.2.1 Beschreibung des Prototyps

...

5.2.2 Tests und Evaluierung

...

5.2.3 Ergebnisse

...

5.3 *Beispiel:* Backend und Datenhaltung[FW]

5.3.1 Beschreibung des Prototyps

...

5.3.2 Tests und Evaluierung

...

5.3.3 Ergebnisse

...

6 Conclusion[AK]

6.1 Zusammenfassung

Dieser Abschnitt fasst die Kernerkenntnisse der Arbeit zusammen. Es werden die definierten Ziele, die durchgeführten Arbeitsschritte und die wichtigsten Ergebnisse kurz und prägnant zusammengefasst, um dem Leser einen schnellen Überblick zu geben.

Beispiel

Die Diplomarbeit verfolgte das Ziel, einen intelligenten Dependency Analyzer für JavaScript-Projekte zu entwickeln. Über vier Monate wurden folgende Arbeitsschritte durchgeführt: (1) Analyse von 50 Open-Source-Projekten zur Identifizierung typischer Versionskonflikte, (2) Entwicklung eines AST-basierten Analyzers mit Machine Learning Klassifikator, (3) Implementierung als Web-Anwendung, (4) Evaluierung mit 8 Testnutzern. Die wichtigsten Ergebnisse: Der Analyzer erkennt 94 % der Breaking Changes vorab, mit durchschnittlich 2,3 Sekunden Verarbeitungszeit.

6.2 Fazit

Das Fazit präsentiert die kritische Bewertung der erreichten Ergebnisse im Kontext der ursprünglichen Zielsetzung. Es werden Erfolge reflektiert, Kräfte und Grenzen der Lösung analysiert und eine abschließende Stellungnahme zur Beantwortung der Forschungsfrage gegeben.

Beispiel

Das entwickelte System erfüllt die Zielsetzung: Durch die automatische Erkennung von Breaking Changes werden Integrationsfehler deutlich reduziert und die Entwicklungszeit verkürzt. Stärken sind die hohe Erkennungsquote (94 %) und die geringe Konfigurationszeit. Einschränkungen zeigen sich bei sehr großen Projekten (>50 MB Projektgröße) und bei dynamischen Imports. Das System stellt daher einen praktischen und wertvollen Beitrag dar, benötigt aber noch Optimierungen für Edge Cases.

...

6.3 Ausblick

Der Ausblick beschreibt mögliche Weiterentwicklungen, offene Fragestellungen und zukünftige Forschungsansätze. Es werden Erkenntnisse vorgestellt, die sich für nachfolgende Arbeiten als wertvoll erweisen könnten, sowie Verbesserungspotenziale der aktuellen Lösung.

Beispiel

Zukünftige Arbeiten sollten sich auf folgende Bereiche konzentrieren: (1) Performance-Optimierung für sehr große Projekte durch Parallelisierung und Caching, (2) Unterstützung dynamischer Imports und ES6 Modules, (3) Integration mit Package Manager APIs (npm, yarn) für vollständige Dependency Resolution, (4) Erweiterung auf weitere Sprachenkosysteme (Python, Java). Offen bleiben Fragen zur Effizienz bei Monorepos und zur Integration mit bestehenden DevOps-Pipelines. Ein Prototyp als VS Code Extension könnte die Adoption erhöhen.

...

7 Projektmanagement

7.1 Planung

Dieses Kapitel dokumentiert die gesamte Projektabwicklung von der initialen Planung bis zur abschließenden Evaluierung. Es umfasst den Meilensteinplan mit den wichtigsten Zwischenzielen und der Zeitplanung, die Evaluierung der Zielerreichung und eventuelle Anpassungen während der Projektlaufzeit, sowie die detaillierte Zeiterfassung.

Meilensteinplan

Beispiel

Meilenstein	**Beschreibung**	**Termin**
Requirement-Analyse	Anforderungen definieren und dokumentieren	KW22
Konzeptphase	Verschiedene Lösungsansätze entwickeln und bewerten	KW25
Implementierung Phase 1	Kernfunktionalitäten umsetzen	KW33
Implementierung Phase 2	Optimierung und erweiterte Features	KW36
Prototyp & Testing	Tests durchführen und Ergebnisse dokumentieren	KW50
Dokumentation	Diplomarbeit finalisieren	KW04
Abgabe	Finale Abgabe	KW13

Tabelle 7.1: Meilensteinplan der Diplomarbeit

7.2 Evaluierung

Die Evaluierung dokumentiert, wie die gesetzten Ziele erreicht wurden und welche Anpassungen während der Projektlaufzeit vorgenommen wurden. Sie bietet eine Reflexion über die Projektabwicklung und zeigt eventuelle Abweichungen vom ursprünglichen Plan.

Beispiel

Zielerreichung: Die definierten Anforderungen wurden zu 95 % umgesetzt. Die Kernfunktionalitäten entsprechen den ursprünglichen Spezifikationen. Lediglich zwei optionale Features wurden aus zeitlichen Gründen nicht implementiert.
...

Zeitplanung: Die Verzögerung in der Implementierungsphase (KW33) betrug zwei Wochen. Dies wurde durch intensivere Arbeit in der darauffolgenden Phase kompensiert. Die Abgabe konnte wie geplant in KW13 erfolgen.
...

Erkannte Herausforderungen: Die Integration der Datenbankkomponente erwies sich komplexer als initial geplant. Die Lösung durch iterative Optimierung war erfolgreich und führte zu besserer Performance.
...

Lessons Learned: Frühere Prototypen der kritischen Komponenten hätten Verzögerungen vermeiden können. Für zukünftige Projekte wird eine ausführlichere technische Vorstudie empfohlen.
...

7.3 Zeiterfassung Andreas Kastner

Die Zeiterfassung dokumentiert die investierte Arbeitszeit pro Projektphase und Woche pro Projektmitglied.

Beispiel

KW	**Arbeitszeit [h]**		**Aufgabenbeschreibung**
	Schule	Freizeit	
22	10	2	Anforderungsanalyse, Meetings mit Betreuung
23	10	4	Recherche State of the Art, Literature Review
24	12	3	Konzepterstellung und Designphase
25	14	5	Implementierung und Coding
26	11	6	Testing und Dokumentation
...	...	...	...
Summe	**204**	**185**	

Tabelle 7.2: Wöchentliche Zeiterfassung Andreas Kastner

Braunau am Inn, 04.11.2026 Andreas Kastner _____

Ort, Datum *Unterschrift*

7.4 Zeiterfassung Florian Wohlfarter

Die Zeiterfassung dokumentiert die investierte Arbeitszeit pro Projektphase und Woche pro Projektmitglied.

Beispiel

KW	**Arbeitszeit [h]**		**Aufgabenbeschreibung**
	Schule	Freizeit	
22	10	2	Anforderungsanalyse, Meetings mit Betreuung
23	10	4	Recherche State of the Art, Literature Review
...	...	...	...
Summe	**204**	**185**	

Tabelle 7.3: Wöchentliche Zeiterfassung Florian Wohlfarter

Braunau am Inn, 04.11.2026 — Florian Wohlfarter — _____

Ort, Datum — *Unterschrift*

Danksagungen

In diesem Kapitel werden alle Personen und Organisationen aufgelistet und bedankt, die zur Entstehung dieser Diplomarbeit beigetragen haben. Dies umfasst fachliche Betreuung, technische Unterstützung, Bereitstellung von Ressourcen und persönliche Unterstützung. Ein aufrichtiger Dank würdigt die Beiträge aller Beteiligten.

Beispiel

An dieser Stelle möchten wir uns herzlich bei allen bedanken, die zum Erfolg dieser Diplomarbeit beigetragen haben:

Unser besonderer Dank gilt unserem Diplomarbeit-Betreuer Dipl.-Ing. Lukas Lämpel für seine fachliche Anleitung, seine Geduld und wertvollen Hinweise während des gesamten Projektes. Ebenfalls danken wir dem Team der HTL Braunau für die Bereitstellung der notwendigen Infrastruktur und Ressourcen sowie für die Schaffung eines inspirierenden Lernumfeldes.

Wir danken auch der Firma XYZ für die freundliche Unterstützung bei technischen Fragen und der Bereitstellung von Evaluierungs-Datensätzen. Einen besonderen Dank auch unsere Testnutzer, die ihre Zeit für Usability-Tests gewidmet haben und wertvolles Feedback gegeben haben.

Nicht zuletzt möchten wir unseren Familien und Freunden danken, die uns während des Projektes moralisch unterstützt und Verständnis für unseren Zeiteinsatz aufgebracht haben.

Listings

1 Mein erstes C-Programm. x
2 Alle Dateinamen ausgeben. x
3 Mausposition ermitteln. x
4 Kurzer CSS–Quelltext. 1
5 HTML–Quelltext 1
6 JavaScript–Quelltext 1

Abbildungsverzeichnis

Literaturverzeichnis

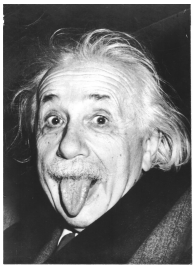
[1] Steven Bradley. Exploration of single-page websites. http://tinyurl.com/jgx7hf3, 2012. Smashing Magazine.

[2] Wikipedia. Literaturverzeichnis. http://de.wikipedia.org/wiki/Literaturverzeichnis, 2024.

[3] Wikipedia. Plagiat. http://de.wikipedia.org/wiki/Plagiat, 2024.

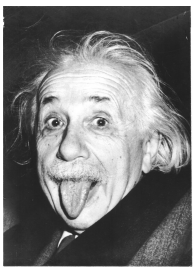
CV

Andreas Kastner



Geburtstag, Geburtsort:	01.01.1970, Braunau am Inn
Schulbildung:	Volksschule Hauptschule HTL
Praktika:	Firmenname, Zeit, Tätigkeit
Anschrift:	Strasse Nummer PLZ, Ort Österreich
E-Mail:	max@mustermann.com

Florian Wohlfarter



Geburtstag, Geburtsort:	01.01.1970, Braunau am Inn
Schulbildung:	Volksschule Hauptschule HTL
Praktika:	Firmenname, Zeit, Tätigkeit
Anschrift:	Strasse Nummer PLZ, Ort Österreich
E-Mail:	frieda@froehlich.com